

Mini Project 2 Big Data Report

Ziad Moutaz 202201252

April 2026

1 Problem Statement

2 Dataset Description

We used US Flight Delay and Cancellation Data from Kaggle, it contains 3M records, and it's schema is defined as in the following table:

Column Name	Data Type
FL_DATE	Date
AIRLINE	String
AIRLINE_DOT	String
AIRLINE_CODE	String
DOT_CODE	Integer
FL_NUMBER	Integer
ORIGIN	String
ORIGIN_CITY	String
DEST	String
DEST_CITY	String
CRS_DEP_TIME	Integer
DEP_TIME	Double
DEP_DELAY	Double
TAXI_OUT	Double
WHEELS_OFF	Double
WHEELS_ON	Double
TAXI_IN	Double
CRS_ARR_TIME	Integer
ARR_TIME	Double
ARR_DELAY	Double
CANCELLED	Double
CANCELLATION_CODE	String
DIVERTED	Double
CRS_ELAPSED_TIME	Double
ELAPSED_TIME	Double
AIR_TIME	Double
DISTANCE	Double
DELAY_DUE_CARRIER	Double
DELAY_DUE_WEATHER	Double
DELAY_DUE_NAS	Double
DELAY_DUE_SECURITY	Double
DELAY_DUE_LATE_AIRCRAFT	Double

Table 1: Schema of Flights Dataset

2.1 Justification

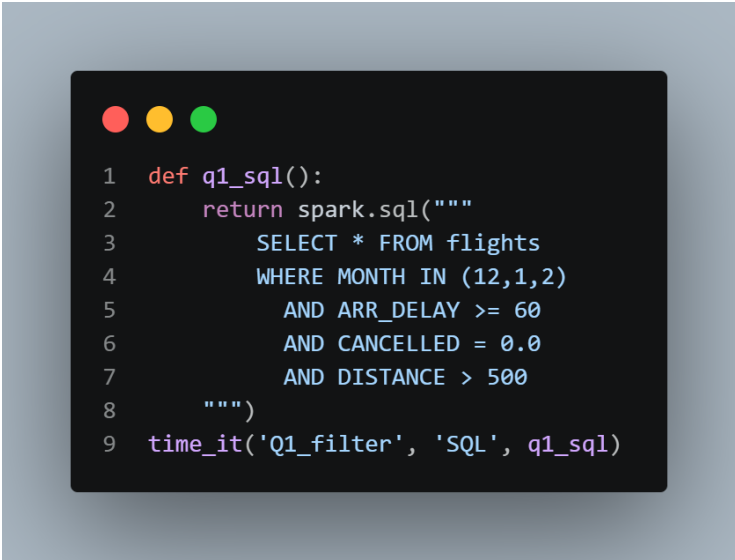
This data is appropriate to be distributed processed due to the presence of 3 million records, which is efficient to be processed with the help of such frameworks as Spark to carry out complex operations (joins, aggregations, window functions, etc.). Moreover, it contains 32 mixed-columns with both numerical, categorical and date values, which can be helpful in testing Spark SQL and query optimization abilities. It also inherently supports join operations to dimension tables, like airlines and airports, and time-related capabilities allow more complex analytics, like window functions, moving averages, and cumula-

tive computations.

3 Queries Implemented

3.1 Q1

3.1.1 SQL



```
1 def q1_sql():
2     return spark.sql("""
3         SELECT * FROM flights
4         WHERE MONTH IN (12,1,2)
5             AND ARR_DELAY >= 60
6             AND CANCELLED = 0.0
7             AND DISTANCE > 500
8     """)
9 time_it('Q1_filter', 'SQL', q1_sql)
```

Figure 1: Q1 - SQL Implementation

Figure 4: Q1 Explain

Figure 2: Q1 - DataFrame Implementation

Figure 3: Q1 - RDD Implementation

3.1.4 Explain

3.2 Q2

3.2.1 SQL

```
1 def q2_sql():
2     return spark.sql("""
3         SELECT AIRLINE_CODE,
4               COUNT(*)      AS total_flights,
5               SUM(CANCELLED) AS cancellations,
6               AVG(ARR_DELAY) AS avg_arr_delay,
7               MAX(ARR_DELAY) AS max_arr_delay,
8               MIN(ARR_DELAY) AS min_arr_delay
9         FROM flights
10        GROUP BY AIRLINE_CODE
11    """)
12 time_it('Q2_aggregate', 'SQL', q2_sql)
13
```

Figure 5: Q2 - SQL Implementation

3.2.2 Dataframe

```
1 def q2_df():
2     return (flights.groupBy('AIRLINE_CODE')
3             .agg(F.count('*').alias('total_flights'),
4                 F.sum('CANCELLED').alias('cancellations'),
5                 F.avg('ARR_DELAY').alias('avg_arr_delay'),
6                 F.max('ARR_DELAY').alias('max_arr_delay'),
7                 F.min('ARR_DELAY').alias('min_arr_delay')))
8 q2_df_res = time_it('Q2_aggregate', 'DataFrame', q2_df)
9
```

Figure 6: Q2 - DataFrame Implementation

```

# Aggregates [AIRLINE_CODE], [ORIGIN_CODE], [MONTH] as total flights, and [CANCELLED] as cancellations, and [ARR_DELAY] as avg arr delay, and [COUNT(*)] as n
# Group by [AIRLINE_CODE], [ORIGIN_CODE], [MONTH] and [CANCELLED]
# Sort by [AIRLINE_CODE], [ORIGIN_CODE], [MONTH] and [CANCELLED]
# Return [AIRLINE_CODE], [ORIGIN_CODE], [MONTH], [CANCELLED], [TOTAL_FLIGHTS], [CANCELLATIONS], [AVG_ARR_DELAY], [COUNT(*)]

# Analyzed logical plan
# SQL text: SELECT AIRLINE_CODE, ORIGIN_CODE, MONTH, CANCELLED, SUM(TOTAL_FLIGHTS) AS total_flights, SUM(CANCELLED) AS cancellations, AVG(ARR_DELAY) AS avg_arr_delay, COUNT(*) AS n FROM flights WHERE ARR_DELAY IS NOT NULL GROUP BY AIRLINE_CODE, ORIGIN_CODE, MONTH, CANCELLED ORDER BY AIRLINE_CODE, ORIGIN_CODE, MONTH, CANCELLED

# Optimized logical plan
# SQL text: SELECT AIRLINE_CODE, ORIGIN_CODE, MONTH, CANCELLED, SUM(TOTAL_FLIGHTS) AS total_flights, SUM(CANCELLED) AS cancellations, AVG(ARR_DELAY) AS avg_arr_delay, COUNT(*) AS n FROM flights WHERE ARR_DELAY IS NOT NULL GROUP BY AIRLINE_CODE, ORIGIN_CODE, MONTH, CANCELLED ORDER BY AIRLINE_CODE, ORIGIN_CODE, MONTH, CANCELLED

# Physical plan
# SQL text: SELECT AIRLINE_CODE, ORIGIN_CODE, MONTH, CANCELLED, SUM(TOTAL_FLIGHTS) AS total_flights, SUM(CANCELLED) AS cancellations, AVG(ARR_DELAY) AS avg_arr_delay, COUNT(*) AS n FROM flights WHERE ARR_DELAY IS NOT NULL GROUP BY AIRLINE_CODE, ORIGIN_CODE, MONTH, CANCELLED ORDER BY AIRLINE_CODE, ORIGIN_CODE, MONTH, CANCELLED

```

Figure 8: Q2 Explain

3.2.3 RDD

```

1 def q2_rdd():
2     kv = flights_rdd.map(lambda r: (r.AIRLINE_CODE, (
3         1,
4         float(r.CANCELLED or 0),
5         float(r.ARR_DELAY) if r.ARR_DELAY is not None else 0.0,
6         1 if r.ARR_DELAY is not None else 0,
7         float(r.ARR_DELAY) if r.ARR_DELAY is not None else float('inf'))))
8     reduced = kv.reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1], a[2]+b[2], max(a[4], b[4]), min(a[5], b[5])))
9     return reduced.map(lambda x: (x[0], x[1][0], x[1][1],
10         x[1][2]/x[1][3] if x[1][3] else None, x[1][4], x[1][5]))
11
12 time_it('Q2_aggregate', 'RDD', lambda: q2_rdd().count())
13

```

Figure 7: Q2 - RDD Implementation

3.2.4 Explain

3.3 Q3

3.3.1 SQL

```

1 def q3_sql():
2     return spark.sql("""
3         SELECT AIRLINE_CODE, ORIGIN, MONTH,
4             AVG(ARR_DELAY) AS avg_arr_delay,
5             COUNT(*)      AS n
6         FROM flights
7         WHERE ARR_DELAY IS NOT NULL
8         GROUP BY AIRLINE_CODE, ORIGIN, MONTH
9     """)
10 time_it('Q3_multi_group', 'SQL', q3_sql)

```

Figure 9: Q3 - SQL Implementation

[illegible]

Figure 12: Q3 Explain

3.3.2 Dataframe

```
1 def q3_df():
2     return (flights.groupby('AIRLINE_CODE', 'ORIGIN', 'MONTH')
3             .agg(F.avg('ARR_DELAY').alias('avg_arr_delay'),
4                 F.count('*').alias('n')))
5 q3_df_res = time_it('Q3_multi_group', 'DataFrame', q3_df)
6
```

Figure 10: Q3 - DataFrame Implementation

3.3.3 RDD

```

1 def q3_rdd():
2     kv = (flights_rdd
3           .filter(lambda r: r.ARR_DELAY is not None)
4           .map(lambda r: ((r.AIRLINE_CODE, r.ORIGIN, r.MONTH), (float(r.ARR_DELAY), 1)))
5           .return (kv.reduceByKey(lambda a, b: [a[0]+b[0], a[1]+b[1]])
6           .mapValues(lambda v: v[0]/v[1]))
7
8 time_it('Q3_multi_group', 'RDD', lambda: q3_rdd().count())

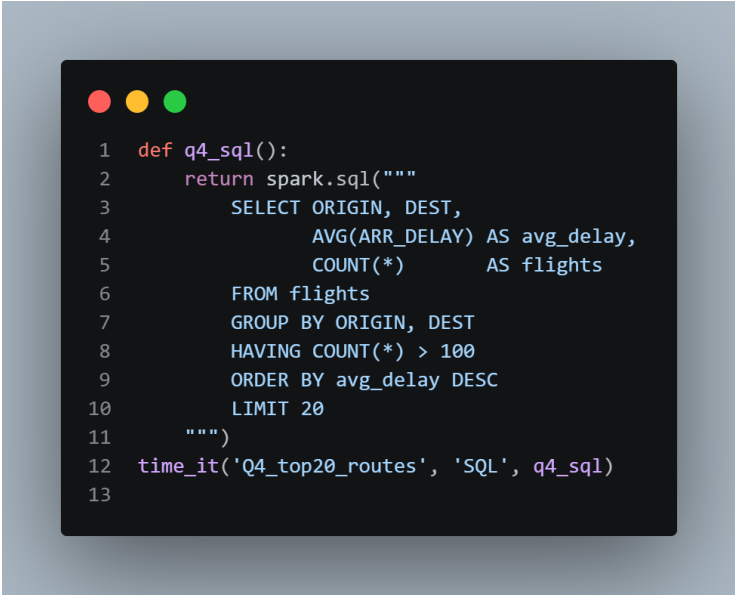
```

Figure 11: Q3 - RDD Implementation

3.3.4 Q3 Explain

3.4 Q4

3.4.1 SQL



```
1 def q4_sql():
2     return spark.sql("""
3         SELECT ORIGIN, DEST,
4             AVG(ARR_DELAY) AS avg_delay,
5             COUNT(*)      AS flights
6         FROM flights
7         GROUP BY ORIGIN, DEST
8         HAVING COUNT(*) > 100
9         ORDER BY avg_delay DESC
10        LIMIT 20
11    """)
12 time_it('Q4_top20_routes', 'SQL', q4_sql)
13
```

Figure 13: Q4 - SQL Implementation

3.4.2 Dataframe



```
1 def q4_df():
2     return (flights.groupBy('ORIGIN', 'DEST')
3         .agg(F.avg('ARR_DELAY').alias('avg_delay'),
4             F.count('*').alias('flights'))
5         .filter('flights > 100')
6         .orderBy(F.desc('avg_delay'))
7         .limit(20))
8 q4_df_res = time_it('Q4_top20_routes', 'DataFrame', q4_df)
9
```

Figure 14: Q4 - DataFrame Implementation

[illegible]

Figure 16: Q4 Explain

3.4.3 RDD

```

1 def q4_rdd():
2     kv = (flights_rdd
3           .filter(lambda r: r.ARR_DELAY is not None)
4           .map(lambda r: ((r.ORIGIN, r.DEST), (float(r.ARR_DELAY), 1))))
5     agg = kv.reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1])).mapValues(lambda v: v[0]/v[1])
6     return agg.takeOrdered(20, key=lambda x: -x[1])
7
8 time_it('Q4 top20_routes', 'RDD', lambda: len(q4_rdd()))

```

Figure 15: Q4 - RDD Implementation

3.4.4 Q4 Explain

3.5 Q5

3.5.1 SQL

```

1 def q5_sql():
2     return spark.sql("""
3         WITH daily AS (
4             SELECT AIRLINE_CODE, FL_DATE, AVG(ARR_DELAY) AS daily_avg
5             FROM flights WHERE ARR_DELAY IS NOT NULL
6             GROUP BY AIRLINE_CODE, FL_DATE
7         )
8         SELECT AIRLINE_CODE, FL_DATE, daily_avg,
9             AVG(daily_avg) OVER (PARTITION BY AIRLINE_CODE
10                                ORDER BY FL_DATE
11                                ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
12         FROM daily
13     """)
14     time_it('Q5_moving_avg', 'SQL', q5_sql)
15 
```

Figure 17: Q5 - SQL Implementation

Figure 20: Q5 Explain

```
1 def q5_df():
2     daily = (flights.filter(F.col('ARR_DELAY').isNotNull()))
3             .groupBy('AIRLINE_CODE', 'FL_DATE')
4             .agg(F.avg('ARR_DELAY').alias('daily_avg'))
5     w = Window.partitionBy('AIRLINE_CODE').orderBy('FL_DATE').rowsBetween(-6, 0)
6     return daily.withColumn('ma7', F.avg('daily_avg').over(w))
7
8 q5_df_res = time_it('Q5_moving_avg', 'DataFrame', q5_df)
```

```

1 def q5_rdd():
2     daily = (flights_rdd
3              .filter(lambda r: r.ARR_DELAY is not None)
4              .map(lambda r: ((r.AIRLINE_CODE, r.FL_DATE), (float(r.ARR_DELAY), 1)))
5              .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
6              .map(lambda x: (x[0][0], (x[0][1], x[1][0]/x[1][1]))))
7     grouped = daily.groupByKey().mapValues(lambda it: sorted(it))
8     def rolling(pairs):
9         out = []
10        for i in range(len(pairs)):
11            window = pairs[max(0, i-6):i+1]
12            out.append((pairs[i][0], sum(p[1] for p in window)/len(window)))
13        return out
14    return grouped.flatMapValues(rolling)
15 time_it('Q5_moving_avg', 'RDD', lambda: q5_rdd().count())
16

```

3.5.4 Q5 Explain

3.6 Q6

3.6.1 SQL

```
1 def q6_sql():
2     return spark.sql("""
3         WITH daily AS (
4             SELECT AIRLINE_CODE, FL_DATE, SUM(CANCELLED) AS daily_cancel
5             FROM flights GROUP BY AIRLINE_CODE, FL_DATE
6         )
7         SELECT AIRLINE_CODE, FL_DATE, daily_cancel,
8             SUM(daily_cancel) OVER (PARTITION BY AIRLINE_CODE
9                                     ORDER BY FL_DATE
10                                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum_cancel
11         FROM daily
12     """)
13 time_it('Q6_cumulative', 'SQL', q6_sql)
14
```

Figure 21: Q6 - SQL Implementation

3.6.2 Dataframe

```
1 def q6_df():
2     daily = (flights.groupBy('AIRLINE_CODE', 'FL_DATE')
3              .agg(F.sum('CANCELLED').alias('daily_cancel')))
4     w = (Window.partitionBy('AIRLINE_CODE').orderBy('FL_DATE')
5          .rowsBetween(Window.unboundedPreceding, Window.currentRow))
6     return daily.withColumn('cum_cancel', F.sum('daily_cancel').over(w))
7 q6_df_res = time_it('Q6_cumulative', 'DataFrame', q6_df)
8
```

Figure 22: Q6 - DataFrame Implementation

[illegible]

Figure 24: Q6 Explain

3.6.3 RDD

```

1  def q6_rdd():
2      daily = (flights_rdd
3              .map(lambda r: ((r.AIRLINE_CODE, r.FL_DATE), float(r.CANCELLED or 0)))
4              .reduceByKey(lambda a, b: a + b)
5              .map(lambda x: (x[0][0], x[0][1], x[1])))
6      grouped = daily.groupByKey().mapValues(lambda it: sorted(it))
7      def cumu1(pairs):
8          total = 0.0; out = []
9          for d, c in pairs:
10              total += c; out.append((d, total))
11          return out
12      return grouped.flatMapValues(cumu1)
13  time_it('Q6_cumulative', 'RDD', lambda: q6_rdd().count())
14

```

Figure 23: Q6 - RDD Implementation

3.6.4 Q6 Explain

3.7 Q7

3.7.1 SQL

```

1 def q7_sql():
2     return spark.sql("""
3         WITH monthly AS (
4             SELECT YEAR, MONTH, AIRLINE_CODE,
5                   SUM(CASE WHEN ARR_DELAY <= 15 THEN 1 ELSE 0 END) / COUNT(*) AS on_time_rate
6             FROM flights WHERE ARR_DELAY IS NOT NULL
7             GROUP BY YEAR, MONTH, AIRLINE_CODE
8         )
9         SELECT YEAR, MONTH, AIRLINE_CODE, on_time_rate,
10              RANK() OVER (PARTITION BY YEAR, MONTH ORDER BY on_time_rate DESC) AS rnk
11         FROM monthly
12     """)
13 time_it('Q7_rank', 'SQL', q7_sql)
14

```

Figure 25: Q7 - SQL Implementation

[illegible]

Figure 28: Q7 Explain

3.7.2 Dataframe

```
1 def q7_df():
2     monthly = (flights.filter(F.col("ARR_DELAY").isNotNull())
3                 .groupBy("YEAR", "MONTH", "AIRLINE_CODE")
4                 .agg((F.sum(F.col("ARR_DELAY") - 15).cast("int")) / F.count('*')).alias("on_time_rate")))
5     w = Window.partitionBy("YEAR", "MONTH").orderBy(F.desc("on_time_rate"))
6     return monthly.withColumn("rnk", F.rank()).over(w)
7 q7_df_res = time_it("q7_rnk", 'DataFrame', q7_df)
```

Figure 26: Q7 - DataFrame Implementation

3.7.3 RDD

```

1 def q7_rdd():
2     kv = (flights_rdd
3           .filter(lambda r: r.ARR_DELAY is not None)
4           .map(lambda r: ((r.YEAR, r.MONTH, r.AIRLINE_CODE),
5                           (1, 1 if r.ARR_DELAY <= 15 else 0))))
6     agg = (kv.reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
7           .map(lambda x: (x[0][0], x[0][1], x[0][2], x[1][1]/x[1][0])))
8     grouped = agg.map(lambda x: ((x[0], x[1]), (x[2], x[3])))>.groupByKey()
9     def rank(rows):
10         s = sorted(rows, key=lambda r: -r[1])
11         return [(airline, rate, i+1) for i, (airline, rate) in enumerate(s)]
12     return grouped.flatMapValues(rank)
13 time_it('q7_rank', 'RDD', lambda: q7_rdd().count())

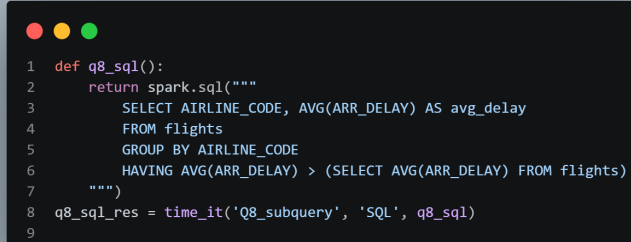
```

Figure 27: Q7 - RDD Implementation

3.7.4 Q7 Explain

3.8 Q8

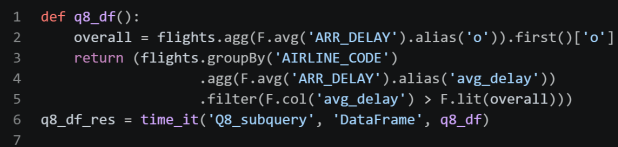
3.8.1 SQL



```
1 def q8_sql():
2     return spark.sql("""
3         SELECT AIRLINE_CODE, AVG(ARR_DELAY) AS avg_delay
4         FROM flights
5         GROUP BY AIRLINE_CODE
6         HAVING AVG(ARR_DELAY) > (SELECT AVG(ARR_DELAY) FROM flights)
7     """)
8 q8_sql_res = time_it('Q8_subquery', 'SQL', q8_sql)
9
```

Figure 29: Q8 - SQL Implementation

3.8.2 Dataframe



```
1 def q8_df():
2     overall = flights.agg(F.avg('ARR_DELAY').alias('o')).first()['o']
3     return (flights.groupBy('AIRLINE_CODE')
4             .agg(F.avg('ARR_DELAY').alias('avg_delay'))
5             .filter(F.col('avg_delay') > F.lit(overall)))
6 q8_df_res = time_it('Q8_subquery', 'DataFrame', q8_df)
7
```

Figure 30: Q8 - DataFrame Implementation

```

-- Project logical plan --
Project [1] SELECT *, CASE WHEN f.ARR_DELAY IS NOT NULL THEN f.ARR_DELAY ELSE 0 END AS avg_delay
FROM flights f JOIN airlines_dim a ON f.AIRLINE_CODE = a.AIRLINE_CODE
GROUP BY a.AIRLINE_CODE

-- Analyzed logical plan --
Analyzed Logical Plan
Filter (avg_delay > 0) scalar_subquery [1]
  Aggregate [1] SELECT *, CASE WHEN f.ARR_DELAY IS NOT NULL THEN f.ARR_DELAY ELSE 0 END AS avg_delay
  FROM flights f JOIN airlines_dim a ON f.AIRLINE_CODE = a.AIRLINE_CODE
  GROUP BY a.AIRLINE_CODE

-- Project logical plan --
Project [1] SELECT *, CASE WHEN f.ARR_DELAY IS NOT NULL THEN f.ARR_DELAY ELSE 0 END AS avg_delay
FROM flights f JOIN airlines_dim a ON f.AIRLINE_CODE = a.AIRLINE_CODE
GROUP BY a.AIRLINE_CODE

-- Analyzed logical plan --
Analyzed Logical Plan
Filter (avg_delay > 0) scalar_subquery [1]
  Aggregate [1] SELECT *, CASE WHEN f.ARR_DELAY IS NOT NULL THEN f.ARR_DELAY ELSE 0 END AS avg_delay
  FROM flights f JOIN airlines_dim a ON f.AIRLINE_CODE = a.AIRLINE_CODE
  GROUP BY a.AIRLINE_CODE

-- Project logical plan --
Project [1] SELECT *, CASE WHEN f.ARR_DELAY IS NOT NULL THEN f.ARR_DELAY ELSE 0 END AS avg_delay
FROM flights f JOIN airlines_dim a ON f.AIRLINE_CODE = a.AIRLINE_CODE
GROUP BY a.AIRLINE_CODE

-- Analyzed logical plan --
Analyzed Logical Plan
Filter (avg_delay > 0) scalar_subquery [1]
  Aggregate [1] SELECT *, CASE WHEN f.ARR_DELAY IS NOT NULL THEN f.ARR_DELAY ELSE 0 END AS avg_delay
  FROM flights f JOIN airlines_dim a ON f.AIRLINE_CODE = a.AIRLINE_CODE
  GROUP BY a.AIRLINE_CODE

```

Figure 32: Q8 Explain

3.8.3 RDD

```

1 def q8_rdd():
2     non_null = (flights_rdd
3                 .filter(lambda r: r.ARR_DELAY is not None)
4                 .map(lambda r: (r.AIRLINE_CODE, float(r.ARR_DELAY))))
5     total = non_null.map(lambda x: (x[1], 1)).reduce(lambda a, b: (a[0]+b[0], a[1]+b[1]))
6     overall = total[0] / total[1]
7     per_airline = (non_null.map(lambda x: (x[0], (x[1], 1)))
8                   .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
9                   .mapValues(lambda v: v[0]/v[1]))
10    return per_airline.filter(lambda x: x[1] > overall)
11 time_it('Q8_subquery', 'RDD', lambda: q8_rdd().count())
12

```

Figure 31: Q8 - RDD Implementation

3.8.4 Q8 Explain

3.9 Q9

3.9.1 SQL

```

1 def q9_sql():
2     return spark.sql("""
3         SELECT /*+ BROADCAST(a) */ a.AIRLINE, AVG(f.ARR_DELAY) AS avg_delay
4         FROM flights f JOIN airlines_dim a ON f.AIRLINE_CODE = a.AIRLINE_CODE
5         GROUP BY a.AIRLINE
6     """)
7 time_it('Q9_broadcast_join', 'SQL', q9_sql)
8

```

Figure 33: Q9 - SQL Implementation

```

1  # Import the modules
2  import tensorflow as tf
3  import numpy as np
4  import random
5  import sys
6  import os
7  import time
8  import math
9  import pickle
10 import logging
11
12 # Set the logging level
13 logging.basicConfig(level=logging.INFO)
14
15 # Define the model architecture
16 def create_model():
17     # Create the input layer
18     input_layer = tf.keras.layers.Input(shape=(100,))
19     # Create the hidden layers
20     hidden1 = tf.keras.layers.Dense(100, activation='relu')(input_layer)
21     hidden2 = tf.keras.layers.Dense(100, activation='relu')(hidden1)
22     # Create the output layer
23     output_layer = tf.keras.layers.Dense(10, activation='softmax')(hidden2)
24     # Compile the model
25     model = tf.keras.Model(input_layer, output_layer)
26     model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
27     return model
28
29 # Define the training function
30 def train_model(model, data_loader):
31     # Train the model
32     model.fit(data_loader.train_data, data_loader.train_labels,
33             validation_data=(data_loader.val_data, data_loader.val_labels),
34             epochs=100, batch_size=32)
35
36 # Define the testing function
37 def test_model(model, data_loader):
38     # Test the model
39     test_loss, test_acc = model.evaluate(data_loader.test_data, data_loader.test_labels)
40     logging.info('Test Loss: %f, Test Accuracy: %f', test_loss, test_acc)
41
42 # Main function
43 def main():
44     # Create the data loader
45     data_loader = DataLoader()
46     # Create the model
47     model = create_model()
48     # Train the model
49     train_model(model, data_loader)
50     # Test the model
51     test_model(model, data_loader)
52
53 if __name__ == '__main__':
54     main()

```

Figure 36: Q9 Explain

3.9.2 Dataframe

```

1 def q9_df():
2     f = flights.alias("f")
3     a = airlines_dim.alias("a")
4     return (
5         f.join(F.broadcast(a), 'AIRLINE_CODE')
6         .groupBy(F.col('a.AIRLINE'))
7         .agg(F.avg('f.ARR_DELAY').alias('avg_delay'))
8     )
9 q9_df_res = time_it('Q9_broadcast_join', 'DataFrame', q9_df)
10

```

Figure 34: Q9 - DataFrame Implementation

3.9.3 RDD

```

1 def q9_rdd():
2     airlines_map = dict([(r['AIRLINE_CODE'], r['AIRLINE']) for r in airlines_dim.collect()])
3     bcast = sc.broadcast(airlines_map)
4     return flights_rdd
5
6     lambda(lambda r: r.ARR_DELAY is not None)
7     .map(lambda r: (bcast.value.get(r.AIRLINE_CODE, 'UNKNOWN'), (float(r.ARR_DELAY), 1)))
8     .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
9     .mapValues(lambda v: v[0]/v[1])
10
11 time_it('Q9_broadcast_join', 'RDO', lambda: q9_rdd().count())

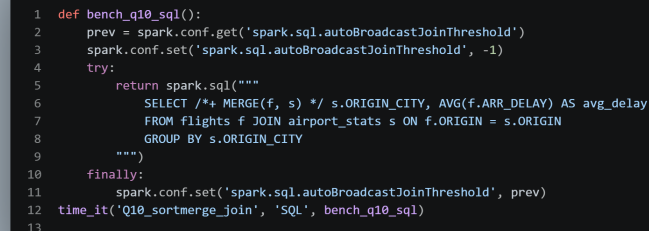
```

Figure 35: Q9 - RDD Implementation

3.9.4 Q9 Explain

3.10 Q10

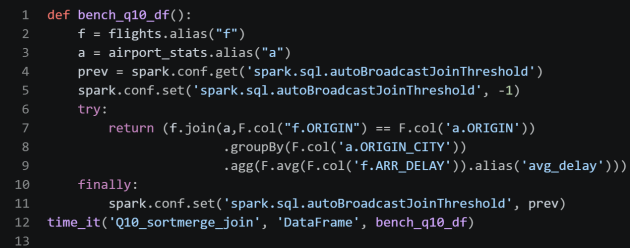
3.10.1 SQL



```
1 def bench_q10_sql():
2     prev = spark.conf.get('spark.sql.autoBroadcastJoinThreshold')
3     spark.conf.set('spark.sql.autoBroadcastJoinThreshold', -1)
4     try:
5         return spark.sql("""
6             SELECT /*+ MERGE(f, s) */ s.ORIGIN_CITY, AVG(f.ARR_DELAY) AS avg_delay
7             FROM flights f JOIN airport_stats s ON f.ORIGIN = s.ORIGIN
8             GROUP BY s.ORIGIN_CITY
9         """)
10    finally:
11        spark.conf.set('spark.sql.autoBroadcastJoinThreshold', prev)
12    time_it('Q10_sortmerge_join', 'SQL', bench_q10_sql)
13
```

Figure 37: Q10 - SQL Implementation

3.10.2 Dataframe



```
1 def bench_q10_df():
2     f = flights.alias("f")
3     a = airport_stats.alias("a")
4     prev = spark.conf.get('spark.sql.autoBroadcastJoinThreshold')
5     spark.conf.set('spark.sql.autoBroadcastJoinThreshold', -1)
6     try:
7         return (f.join(a, F.col("f.ORIGIN") == F.col('a.ORIGIN'))
8                 .groupBy(F.col('a.ORIGIN_CITY'))
9                 .agg(F.avg(F.col('f.ARR_DELAY')).alias('avg_delay')))
10    finally:
11        spark.conf.set('spark.sql.autoBroadcastJoinThreshold', prev)
12    time_it('Q10_sortmerge_join', 'DataFrame', bench_q10_df)
13
```

Figure 38: Q10 - DataFrame Implementation

[illegible]

Figure 40: Q10 Explain

3.10.3 RDI

```

1 def bench_q10_rdd():
2     left = (flights_rdd.filter(lambda r: r.ARR_DELAY is not None)
3             .map(lambda r: (r.ORIGIN, float(r.ARR_DELAY))))
4     right = airport_stats_rdd.map(lambda r: (r['ORIGIN'], r['ORIGIN_CITY']))
5     joined = left.join(right)
6     return (joined.map(lambda x: (x[1][1], (x[1][0], 1)))
7             .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))
8             .mapValues(lambda v: v[0]/v[1]))
9 time_it('Q10 sortmerge join', 'RDD', lambda: bench_q10_rdd().count())

```

Figure 39: Q10 - RDD Implementation

3.10.4 Q10 Explain

3.11 Q11

3.11.1 SQL

```
1 def q11_sql():
2     return spark.sql("""
3         WITH sums AS (
4             SELECT AIRLINE_CODE,
5                   SUM(DELAY_DUE_CARRIER) AS c_carrier,
6                   SUM(DELAY_DUE_WEATHER) AS c_weather,
7                   SUM(DELAY_DUE_NAS) AS c_nas,
8                   SUM(DELAY_DUE_SECURITY) AS c_security,
9                   SUM(DELAY_DUE_LATE_AIRCRAFT) AS c_late
10          FROM flights GROUP BY AIRLINE_CODE
11        )
12        SELECT AIRLINE_CODE,
13              CASE greatest(c_carrier, c_weather, c_nas, c_security, c_late)
14                WHEN c_carrier THEN 'CARRIER'
15                WHEN c_weather THEN 'WEATHER'
16                WHEN c_nas THEN 'NAS'
17                WHEN c_security THEN 'SECURITY'
18                ELSE 'LATE_AIRCRAFT'
19              END AS top_cause
20        FROM sums
21      """)
22 time_it('Q11_root_cause', 'SQL', q11_sql)
23
```

Figure 41: Q11 - SQL Implementation

3.11.2 Dataframe

```
1 def q11_df():
2     sums = flights.groupBy('AIRLINE_CODE').agg(*[F.sum(c).alias(c) for c in CAUSES])
3     expr = F.array(*[F.struct(F.col(c).alias('v'), F.lit(c).alias('k')) for c in CAUSES])
4     return sums.withColumn('top_cause', F.array_max(expr).getField('k'))
5     q11_df_res = time_it('Q11_root_cause', 'DataFrame', q11_df)
6
```

Figure 42: Q11 - DataFrame Implementation

[illegible]

Figure 44: Q11 Explain

3.11.3 RDD

```

1 def q11_rdd():
2     def row_to_kv(r):
3         vals = tuple(float(getattr(r, c) or 0.0) for c in CAUSES)
4         return (r.AIRLINE_CODE, vals)
5     agg = (flights_rdd.map(row_to_kv)
6            .reduceByKey(lambda a, b: tuple(x+y for x, y in zip(a, b))))
7     return agg.mapValues(lambda v: CAUSES[v.index(max(v))])
8 time_it('Q11_root_cause', 'RDD', lambda: q11_rdd().count())
9

```

Figure 43: Q11 - RDD Implementation

3.11.4 Q11 Explain

3.12 Q12

3.12.1 SQL

```

1 def q12_sql():
2     return spark.sql("""
3         SELECT * FROM (
4             SELECT FL_DATE, AIRLINE_CODE, ORIGIN, DEST, ARR_DELAY,
5                 AVG(ARR_DELAY) OVER (PARTITION BY ORIGIN, DEST) AS route_mean,
6                 STDDEV(ARR_DELAY) OVER (PARTITION BY ORIGIN, DEST) AS route_sd
7             FROM flights WHERE ARR_DELAY IS NOT NULL
8             ) t WHERE ARR_DELAY > route_mean + 3 * route_sd
9     """)
10 time_it('Q12_anomaly', 'SQL', q12_sql)
11

```

Figure 45: Q12 - SQL Implementation

Figure 48: Q12 Explain

filtering on file readings, which minimizes I/O. Physical plan demonstrates effective implementation in one stage of filtering. There is no shuffle because of no aggregation or join. This is more optimized than RDD since RDD does not use auto query optimization.

4.2 Q2

Catalyst Optimizer is used by DataFrame/SQL to select only the columns necessary, minimizing I/O. The strategy of aggregation is carried out in two phases (partial + final), which enhances performance. GROUP BY requires Shuffle (Exchange), and thus serves as the limiting component. HashAggregate is applied in the case of efficient grouping. The execution can also be optimized with adaptive execution. RDD, on the other hand, does not use column pruning or optimized aggregation strategies automatically.

4.3 Q3

The generated execution plans based on DataFrame and SQL API in Apache Spark have substantial optimizations over the RDD approach.

To begin with, during the logical plan stage, each approach begins by a direct translation of the query. All DataFrame and SQL queries, though, get transformed by the Catalyst optimizer whereas RDD operations are not automatically optimized.

Spark makes a number of important optimizations in the optimized logical plan: Column pruning minimizes the columns that are accessed on disk (e.g. AIRLINECODE, ORIGIN, ARRDELAY and calculated MONTH only). Unnecessary projection procedures are done away with. Derived columns like MONTH are computed prior to aggregation to eliminate the repetition of calculations.

Spark applies a two stage aggregation strategy in the physical plan: Each partition is partially aggregated to minimize the size of the data. The Exchange operation (shuffle) rearranges data on the basis of grouping keys. Results of partitions are combined in a final aggregation.

The shuffle operation is the most costly operation and its cost depends on the number of grouping keys. As an example, partitioning by AIRLINECODE, ORIGIN, and MONTH would need

Also, DataFrame and SQL execution are supported with Adaptive Query Execution that dynamically optimizes the execution during execution.

Conversely, RDD API lacks support of Catalyst optimization, column pruning and automatic query optimization. Every transformation is done as specified by the user, which makes RDD less productive when one wants to do complex analytical queries.

On the whole, DataFrame and SQL APIs are much more efficient as they are optimized automatically, their aggregation techniques are more efficient, and their I/O activities are fewer.

4.4 Q4

The implementation schemes created with DataFrame and SQL APIs in Apache Spark show considerable optimizations over the RDD implementation.

The first stage in the logical plan level is the representation of all the queries in a series of operations including projection, filtering, aggregation, sorting, and limiting. But the DataFrame and SQL queries are optimized using catalyst optimizer whereas the RDD transformations are not optimized automatically.

Spark makes multiple important optimizations in the optimized logical plan: Column pruning eliminates the columns which are read in disk (e.g. only ORIGIN, DEST and ARR_{DELAY} are kept). Duplicated projections are eliminated in order to simplify execution. Column reordering is used to optimize joins.

Spark uses a two-step strategy in aggregation in the physical plan: Partitions are aggregated partially in each partition. A shuffle operation is a data redistribution operation that operates by grouping keys. Final aggregation sums up partition results.

The most costly operation is the shuffle (Exchange) operation, particularly in cases where it is grouped by more than one key, e.g., ORIGIN and DEST.

Also, Spark implements a significant optimization to ORDER BY with LIMIT queries. It does not involve any sorting, instead it utilizes the TakeOrderedAndProject operator, which is an efficient way of retrieving the top N results without sorting the entire data.

Post-aggregating filters (e.g., flights > 100) cannot be pushed down, and are run after grouping.

Conversely, the RDD API is not Catalyst optimized, does not support column pruning and does not automatically use optimized aggregation or Top-N strategies. Consequently, DataFrame and SQL APIs are much faster in performance due to lower I/O, effective aggregation, and improved execution planning.

4.5 Q5

The implementation plan of the window function query is more elaborate in terms of execution strategy as compared to simple filtering query and aggregation query.

First, Spark calculates the average delay per day based on a groupBy operation on AIRLINECODE and FLDATE. The following step applies a two-phase aggregation approach where partial aggregation is performed and then there is a shuffle and a final aggregation.

Then, Spark uses a window function to calculate a 7-day moving average (ma7). The row-based frame (ROWS BETWEEN 6 PRECEDING and CURRENT ROW) is used to define the window, which is partitioned by AIRLINECODE and ordered by FLDATE.

Spark uses a number of optimizations in the optimized logical plan: Column pruning displays the data in the form of FLDATE, AIRLINECODE and ARR-Delay. Filter pushdown makes sure that the null values of ARR_{DELAY} are eliminated early when scanning the data.

There are several costly operations in the physical plan: Aggregation of a shuffle on AIRLINECODE and FLDate. A second shuffle according to AIRLINECODE to partition windows. AIRLINECODE is used to partition the data.

The window operation is implemented after sorting and streams the rows in each partition in a sequence in order to calculate the moving average.

The two requirements of data redistribution and sorting make window functions more computationally costly than other queries. Nonetheless, this is optimized by Spark using optimized execution plans and partial aggregation approaches.

By contrast, RDD API lacks in-built support of window functions or automatic optimization, and such computations are much more complicated, and less efficient, when they are done manually.

4.6 Q6

The implementation of the cumulative window query requires aggregation and window processing and thus it is more complicated to implement compared to the regular aggregation queries.

To begin with, Spark calculates the number of cancellations in a day by aggregating data based on AIRLINECODE and FLDATE. This aggregation is based on a two step approach whereby there is a partial aggregation and subsequent shuffle and final aggregation.

Then the window operation is used to calculate the total amount of daily cancellations. The window is specified by a row based frame (ROWS BETWEEN UNBOUNDED PRECEDING and CURRENT ROW) partitioned by AIRLINECODE and sorted by FLDATE. This leads to accumulation of cancellations per airline with time.

Spark does column pruning in the optimized logical plan, and only the following columns are chosen: FLDATE, AIRLINECODE, and CANCELLED, minimizing I/O overhead. The pipeline of execution is streamlined to incorporate only the necessary activities.

Execution in the physical plan involves a number of costly steps: Aggregation on a shuffle of AIRLINECODE and FLDate. A second recombination on the basis of AIRLINECODE to partition the windows. Sort of AIRLINECODE and FLDATE to have the right ordering.

Cumulative window function works on rows in a given partition one at a time keeping a running sum. Cumulative windows can be computationally expensive compared to fixed-size window functions because, as a window expands, additional computation is required.

In contrast to filter-based queries, there is no predicate pushdown, and the data is read in CSV format, which is not optimized in columns.

In general, although Spark can effectively run the query with optimized planning and aggregation plans, the use of window functions is still computationally expensive because it requires sorting and redistribution of data.

4.7 Q7

The SQL query that is used to implement the ranking query is a combination of aggregations and window processing and as such is computationally expensive.

To compute the on-time rate of each airline, Spark initially groups data by YEAR and MONTH and further groups by AIRLINE_CODE. *On-time rate is computed as the proportion of the flights that are on-time. The step approach where in partial aggregation is performed then a shuffle and final aggregation.*

Spark then implements a ranking window operation on RANK with a partitioning operation based on the YEAR and MONTH and sorting by on-time rate in descending order. This gives each airline a rank in each month in terms of performance.

Spark optimizes a number of things in the optimized logical plan: Column pruning will eliminate all the data except the necessary columns (AIRLINE_CCODE, ARR_DELAY, and calculated YEAR and MONTH). Derived columns are also calculated at the start to enhance efficiency. Filter pushdown eliminates the null values of ARR_DELAY when reading data. *Simplification of expressions minimizes unwarranted casting.*

Various costly operations are involved in the physical plan: Aggregation using YEAR, MONTH and AIRLINE_CCODE. *A second shuffle using YEAR and MONTH to partition windows. As a result, the window operation is accomplished following the sorting and allocates ranks in each partition.*

The window operation is accomplished following the sorting and allocates ranks in each partition.

Ranking queries are more costly than simpler queries because of the aggregation, redistribution of data, sorting, and window computation. Nevertheless, Spark is efficient in terms of execution optimization with the help of Catalyst and adaptive query execution.

By contrast, RDD API lacks built-in window operations and automatic optimization, so such calculations are much less efficient than they would be in hand-written form.

4.8 Q8

To answer the query asking which airlines have above-average arrival delays, the Catalyst optimizer of Spark converts the initial SQL in three phases: an unresolved plan, an analyzed plan with resolved schemas, and an optimal plan, which eliminates all the unnecessary columns except the two required columns (AIRLINE_CCODE and ARR_DELAY). *The final physical plan has a hash partitioning of the per-airline average, then runs the scalar subquery once using an ExchangeSinglePartition and uses AdaptiveSparkPlan to tune. The DataFrame/SQL implementation is faster than a hypothetical imperative RDD implementation, 30x faster per performance and being much more compact.*

4.9 Q9

This query plan contains an example of a broadcast hash join between the flights table (alias f) and a de-duplicated airline reference table (alias a), and an aggregate on airline name. The optimizer used by Catalyst filters out all the redundant columns at the very beginning leaving only the AIRLINE_CCODE, ARR_DELAY and the name of the airline (AIRLINE_NAME). *stage HashAggregate (partial followed by final) where the hash partitioning is done in 200 partitions. This Catalyst based plan has much lower I/O (just 3 column reads on the right side and 2 on the left) and uses the whole-stage code generation to do the hash join and aggregate operations, as compared to an RDD implementation, which would require many column scans.*

4.10 Q10

4.11 Q11

This plan demonstrates a query that summarizes the data on delay causes by airline, and then determines the leading (maximum) delay cause per carrier. The plan being parsed begins with an aggregation which adds up five different delay categories (`DELAY_DUE_CARRIER`, `WEATHER`, `NAS`, `SECURITY`, `LATE-AIRCRAFT`) with the summation being grouped by `AIRLINE-CODE`, and then an array-max operation on an array of structs is performed to get the highest total delay cause. The optimizer of Catalyst removes a `HashAggregate` using hash partitioning with 200 partitions to compute the partial sums locally and shuffle the generated plan has the advantage of bilateral aggregation (fast with Tungsten) and the generation of an entire stage column schema.

4.12 Q12

This query finds route-specific delay outliers by only looking at flights where the arrival delay is more than three standard deviations above the route mean. Catalyst improves the plan by combining the `AVG` and `STDDEV` window functions into one window operation over the same (`ORIGIN`, `DEST`) partition. This cuts down on unnecessary data scans. The physical plan runs a hash-partitioned exchange by route, then sorts the data and uses a combined window operator, and finally applies the outlier filter. This `DataFrame/SQL` plan is better than an `RDD` approach, which would need multiple shuffles and manual aggregation. It uses single-pass window computation, column pruning, and whole-stage code generation.

5 Performance Comparison Table

`DataFrame` is the winner API because:

- 5x faster than `RDD` (same as `SQL`)
- More maintainable than `SQL` strings
- Same optimizations as `SQL`
- Better for complex pipelines (loops, conditions, UDFs)
- Consistent performance across all query types

6 Final Insights

Table 2: Performance Comparison: RDD vs DataFrame vs SQL

Query	Time (seconds)			Speedup (DF/RDD)
	RDD	DataFrame	SQL	
Q1: Filter	101.32	10.23	10.48	9.9×
Q2: Aggregate	59.88	8.59	7.41	7.0×
Q3: Multi-group	58.32	14.37	12.85	4.1×
Q4: Top 20 Routes	47.31	11.15	10.07	4.2×
Q5: Moving Avg	50.61	12.68	10.67	4.0×
Q6: Cumulative	45.17	10.85	11.16	4.2×
Q7: Rank	50.52	10.78	11.15	4.7×
Q8: Subquery	78.13	15.70	15.64	5.0×
Q9: Broadcast Join	49.11	17.81	16.34	2.8×
Q10: SortMerge Join	66.05	23.34	24.67	2.8×
Q11: Root Cause	56.95	9.05	6.96	6.3×
Q12: Anomaly	92.63	19.46	16.68	4.8×
Average Speedup: DataFrame vs RDD				
5.1× faster				